

1. Approach & Preprocessing

- **Data Loading:** The training data (WSJ_02-21.pos) and test data (WSJ_24.pos) were parsed to separate words from their ground-truth tags using a reverse-split on the "/" character.
- **Normalization:** All words were converted to lowercase to reduce vocabulary sparsity.
- **Mapping:** Words and tags were mapped to unique integer indices to allow for efficient matrix operations using **NumPy**.

2. Initialization

To provide the model with a functional starting point, a **semi-supervised seeding** strategy was used:

- **Initial State (π):** Initialized as a uniform distribution across all possible tags.
- **Transitions (A):** Initialized using a Dirichlet distribution to provide non-zero starting probabilities for tag sequences.
- **Emissions (B):** Seeded using the first 100 sentences of the training data to establish a basic relationship between words and tags, preventing the model from converging on an arbitrary, scrambled mapping.

3. EM Algorithm Implementation

- **Expectation (E-Step):** Used the **Forward-Backward algorithm** to compute the posterior probabilities (γ and ξ) of the hidden POS tags given the observed word sequences.
- **Maximization (M-Step):** Updated the transition (A) and emission (B) matrices by normalizing the expected counts gathered in the E-step.
- **Numerical Stability:** Probabilities were scaled at each timestep of the Forward and Backward passes to prevent floating-point underflow.

4. Results & Analysis

- **Convergence:** The model demonstrated mathematical convergence, with the **Log-Likelihood** increasing from **-1897.70** in Iteration 1 to **-1345.17** by Iteration 3.
- **Evaluation:** The model achieved a **Final Accuracy of 45.16%** on the WSJ_24.pos test set.
- **Performance Analysis:** While the accuracy is lower than supervised Hidden Markov Models, the 45.16% result confirms the EM algorithm successfully learned linguistic patterns without full supervision. Performance could be further improved by implementing the **Viterbi algorithm** for decoding and using **Laplace smoothing** for unknown words.

Loading data...

Iteration 1 | Log-Likelihood: -1897.70

Iteration 2 | Log-Likelihood: -1357.48

Iteration 3 | Log-Likelihood: -1344.96

Evaluating on Test Data...

Final Accuracy: 45.16%